Josizal University
80 Shaw Boulevard, Mandaluyong City
College Division, Computer Science and Engineering

Python Laboratory Activity: List and Tuples

Problem 1: Student Grade Analysis System

Requirements:

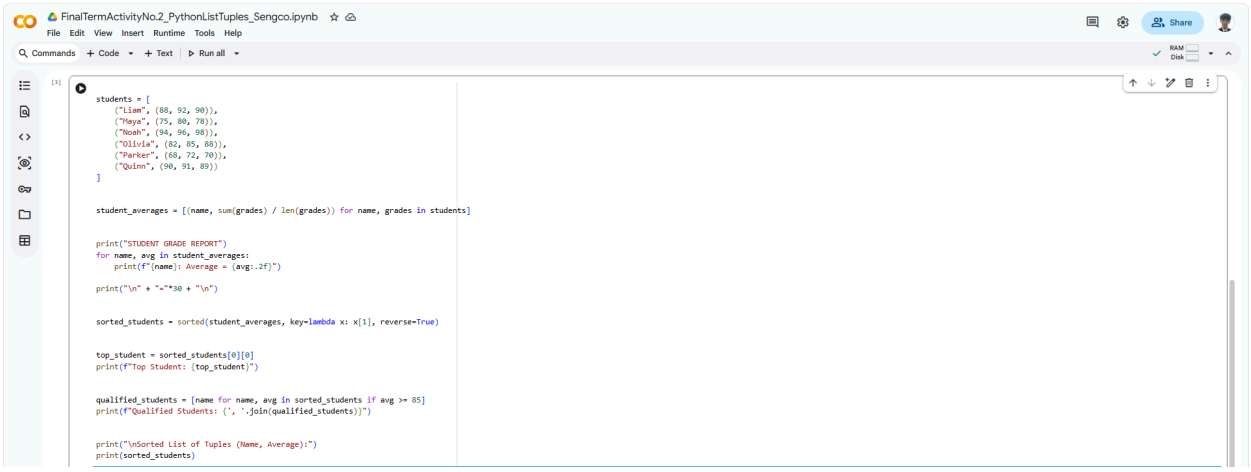
1. Calculate each student's average grade.
2. Display the name and average of every student.
3. Identify the student with the highest average.
4. Display all students whose average is at least 85.
5. Sort the students from highest to lowest average without modifying the original list.
6. Convert the sorted results into a list of tuples containing the student's name and average.



Problem 1: Source Code



Problem 1: Program Output



Problem 1: Additional Test Case Source Code



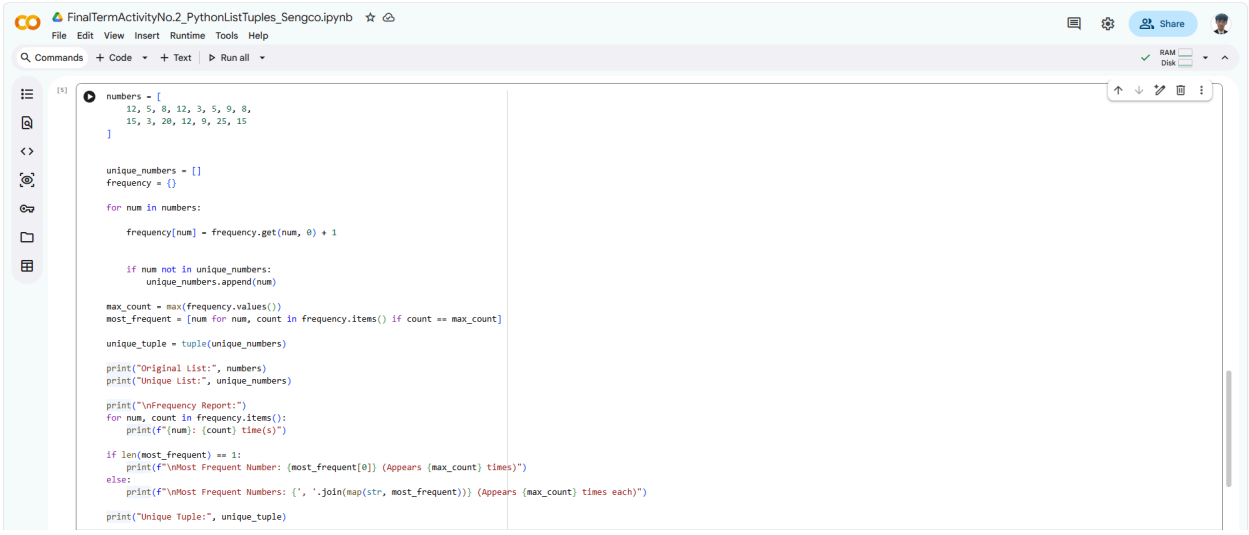
Problem 1: Additional Test Case Program Output

Problem 2: Duplicate Removal Without Losing Order

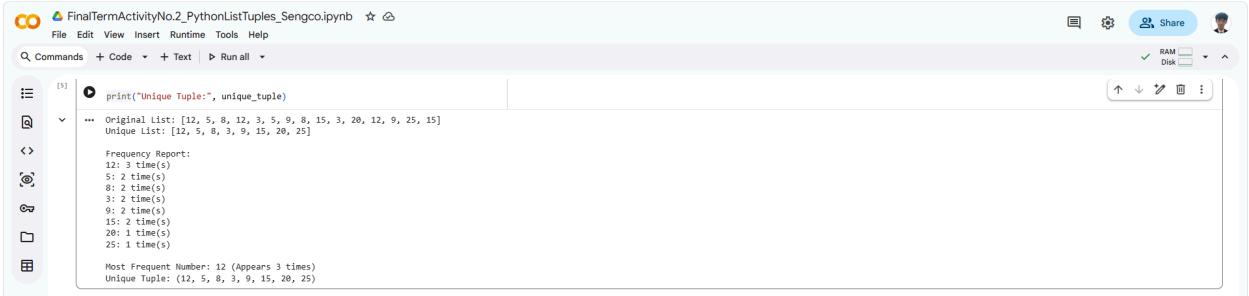
Requirements:

1. Create a new list containing only unique values.
2. Do not use `set()` to remove duplicates.
3. Count how many times each number appears.
4. Identify the most frequently occurring number or numbers.
5. Convert the unique list into a tuple.
6. Display the original list, unique list, frequency report, and tuple.

Challenge: If multiple numbers have the same highest frequency, display all of them.



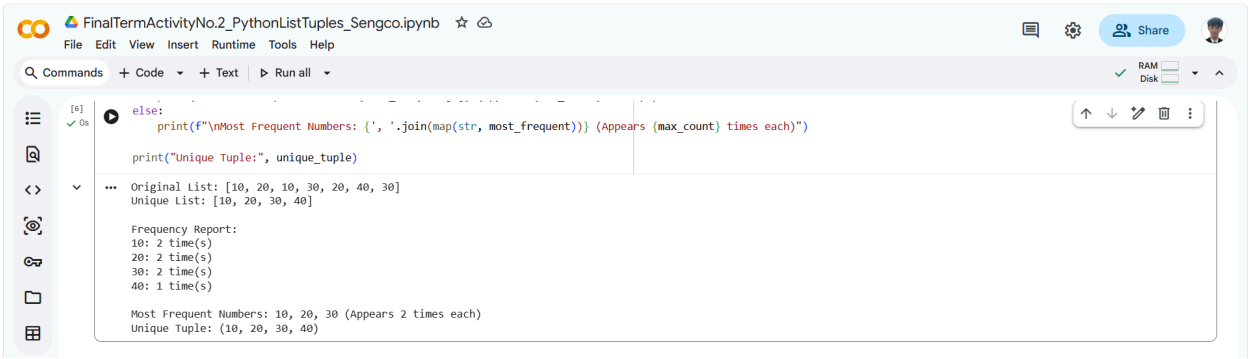
Problem 2: Source Code



Problem 2: Program Output



Problem 2: Additional Test Case Source Code



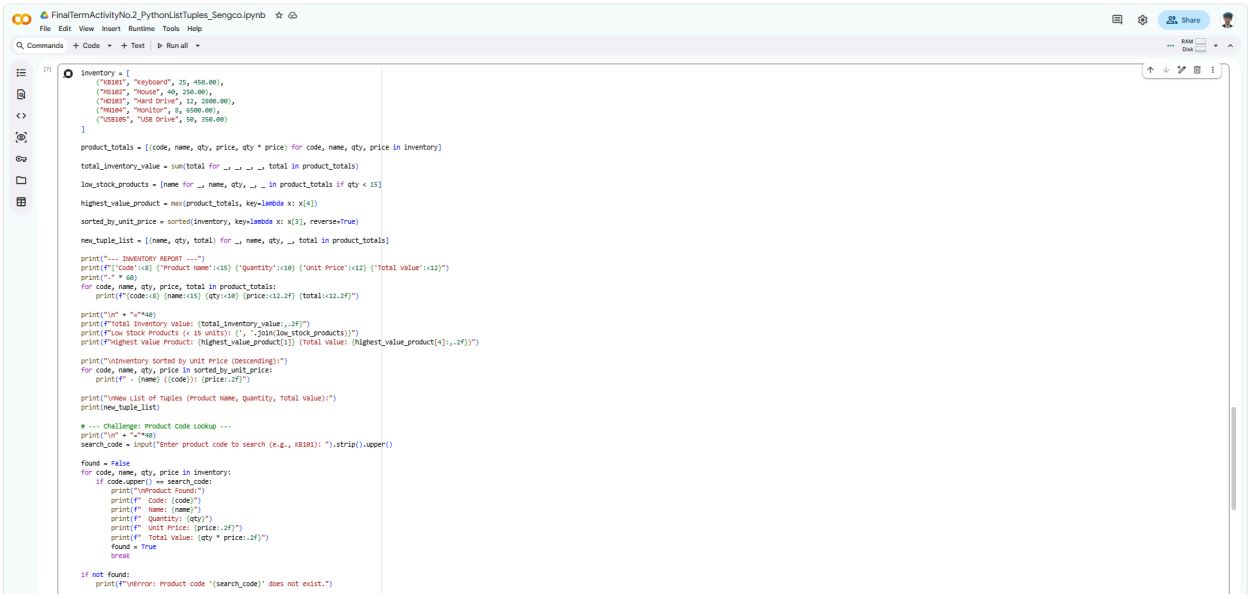
Problem 2: Additional Test Case Program Output

Problem 3: Tuple-Based Inventory Management

Requirements:

1. Calculate the total value of each product using quantity multiplied by unit price.
2. Calculate the total inventory value.
3. Identify products with fewer than 15 units.
4. Find the product with the highest total inventory value.
5. Sort the inventory by unit price in descending order.
6. Generate a new list of tuples containing product name, quantity, and total value.

Challenge: Allow the user to enter a product code and display the matching product information. Display an appropriate message if the code does not exist.



```
inventory = [
    ("KB101", "Keyboard", 25, 450.00),
    ("MS102", "Mouse", 40, 250.00),
    ("HD103", "Hard Drive", 12, 2800.00),
    ("MN104", "Monitor", 8, 6500.00),
    ("USB105", "USB Drive", 50, 350.00)
]

product_totals = [(code, name, qty, price, qty * price) for code, name, qty, price in inventory]

total_inventory_value = sum(total for _, _, _, _, total in product_totals)

low_stock_products = [name for _, name, qty, _, _ in product_totals if qty < 15]

highest_value_product = max(product_totals, key=lambda x: x[4])

sorted_by_unit_price = sorted(inventory, key=lambda x: x[3], reverse=True)

new_tuple_list = [(name, qty, total) for _, name, qty, _, total in product_totals]

print("--- INVENTORY REPORT ---")
print(f"{'Code':<8} {'Product Name':<15} {'Quantity':<10} {'Unit Price':<12} {'Total Value':<12}")
print("-" * 60)
for code, name, qty, price, total in product_totals:
    print(f"{code:<8} {name:<15} {qty:<10} {price:<12.2f} {total:<12.2f}")

print("\n" + "="*40)
print(f"Total Inventory Value: {total_inventory_value:,.2f}")
print(f"Low Stock Products (< 15 units): {', '.join(low_stock_products)}")
print(f"Highest Value Product: {highest_value_product[1]} (Total Value: {highest_value_product[4]:,.2f})")

print("\nInventory Sorted by Unit Price (Descending):")
for code, name, qty, price in sorted_by_unit_price:
    print(f" - {name} ({code}): {price:.2f}")

print("\nNew List of Tuples (Product Name, Quantity, Total Value):")
print(new_tuple_list)

# --- Challenge: Product Code Lookup ---
print("\n" + "="*40)
search_code = input("Enter product code to search (e.g., KB101): ").strip().upper()

found = False
for code, name, qty, price in inventory:
    if code.upper() == search_code:
        print("\nProduct Found:")
        print(f"  Code: {code}")
        print(f"  Name: {name}")
        print(f"  Quantity: {qty}")
        print(f"  Unit Price: {price:.2f}")
        print(f"  Total Value: {qty * price:.2f}")
        found = True
        break

if not found:
    print(f"\nError: Product code '{search_code}' does not exist.")
```

Problem 3: Source Code



```
--- INVENTORY REPORT ---
Code     Product Name    Quantity   Unit Price   Total Value
------------------------------------------------------------
KB101    Keyboard        25         450.00       11250.00
MS102    Mouse           40         250.00       10000.00
HD103    Hard Drive      12         2800.00      33600.00
MN104    Monitor         8          6500.00      52000.00
USB105   USB Drive       50         350.00       17500.00

========================================
Total Inventory Value: 124,350.00
Low Stock Products (< 15 units): Hard Drive, Monitor
Highest Value Product: Monitor (Total Value: 52,000.00)

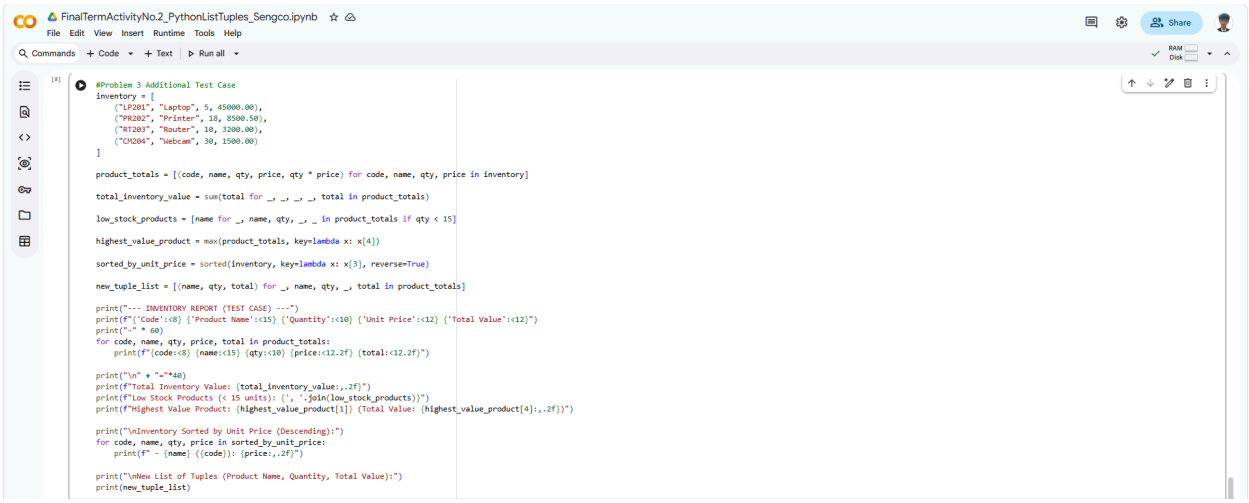
Inventory Sorted by Unit Price (Descending):
 - Monitor (MN104): 6500.00
 - Hard Drive (HD103): 2800.00
 - Keyboard (KB101): 450.00
 - USB Drive (USB105): 350.00
 - Mouse (MS102): 250.00

New List of Tuples (Product Name, Quantity, Total Value):
[('Keyboard', 25, 11250.0), ('Mouse', 40, 10000.0), ('Hard Drive', 12, 33600.0), ('Monitor', 8, 52000.0), ('USB Drive', 50, 17500.0)]

========================================
Enter product code to search (e.g., KB101): MS102

Product Found:
  Code: MS102
  Name: Mouse
  Quantity: 40
  Unit Price: 250.00
  Total Value: 10000.00
```

Problem 3: Program Output



Problem 3: Additional Test Case Source Code

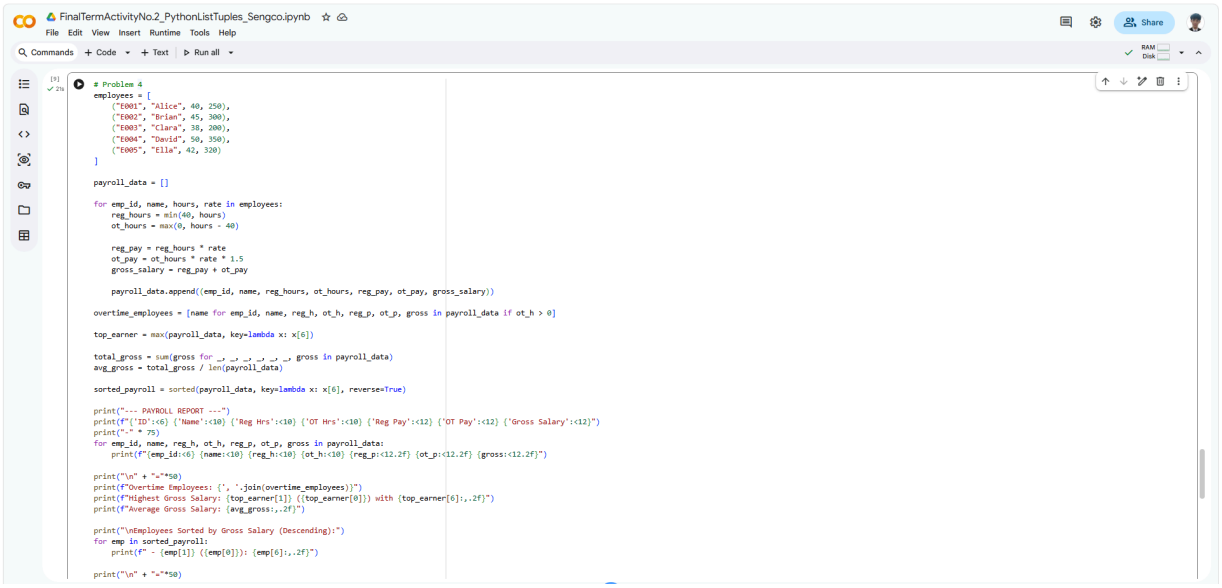


Problem 3: Additional Test Case Program Output

Problem 4: Tuple-Based Employee Payroll Analysis

Requirements:

1. Calculate regular pay and overtime pay for each employee.
2. Calculate the gross salary of each employee.
3. Display employees who worked overtime.
4. Identify the employee with the highest gross salary.
5. Calculate the average gross salary.
6. Store the results as tuples in a new list.
7. Sort employees by gross salary from highest to lowest.



```
# Problem 4
employees = [
    ("E001", "Alice", 40, 250),
    ("E002", "Brian", 45, 300),
    ("E003", "Clara", 38, 200),
    ("E004", "David", 50, 350),
    ("E005", "Ella", 42, 320)
]

payroll_data = []

for emp_id, name, hours, rate in employees:
    reg_hours = min(40, hours)
    ot_hours = max(0, hours - 40)

    reg_pay = reg_hours * rate
    ot_pay = ot_hours * rate * 1.5
    gross_salary = reg_pay + ot_pay

    payroll_data.append((emp_id, name, reg_hours, ot_hours, reg_pay, ot_pay, gross_salary))

overtime_employees = [name for emp_id, name, reg_h, ot_h, reg_p, ot_p, gross in payroll_data if ot_h > 0]

top_earner = max(payroll_data, key=lambda x: x[6])

total_gross = sum(gross for _, _, _, _, _, _, gross in payroll_data)
avg_gross = total_gross / len(payroll_data)

sorted_payroll = sorted(payroll_data, key=lambda x: x[6], reverse=True)

print("--- PAYROLL REPORT ---")
print(f"{'ID':<6} {'Name':<10} {'Reg Hrs':<10} {'OT Hrs':<10} {'Reg Pay':<12} {'OT Pay':<12} {'Gross Salary':<12}")
print("-" * 75)
for emp_id, name, reg_h, ot_h, reg_p, ot_p, gross in payroll_data:
    print(f"{emp_id:<6} {name:<10} {reg_h:<10} {ot_h:<10} {reg_p:<12.2f} {ot_p:<12.2f} {gross:<12.2f}")

print("\n" + "="*50)
print(f"Overtime Employees: {', '.join(overtime_employees)}")
print(f"Highest Gross Salary: {top_earner[1]} ({top_earner[0]}) with {top_earner[6]:,.2f}")
print(f"Average Gross Salary: {avg_gross:,.2f}")

print("\nEmployees Sorted by Gross Salary (Descending):")
for emp in sorted_payroll:
    print(f" - {emp[1]} ({emp[0]}): {emp[6]:,.2f}")

print("\n" + "="*50)
```

Problem 4: Source Code

```
--- PAYROLL REPORT ---
ID     Name       Reg Hrs    OT Hrs     Reg Pay      OT Pay       Gross Salary
---------------------------------------------------------------------------
E001   Alice      40         0          10000.00     0.00         10000.00
E002   Brian      40         5          12000.00     2250.00      14250.00
E003   Clara      38         0          7600.00      0.00         7600.00
E004   David      40         10         14000.00     5250.00      19250.00
E005   Ella       40         2          12800.00     960.00       13760.00

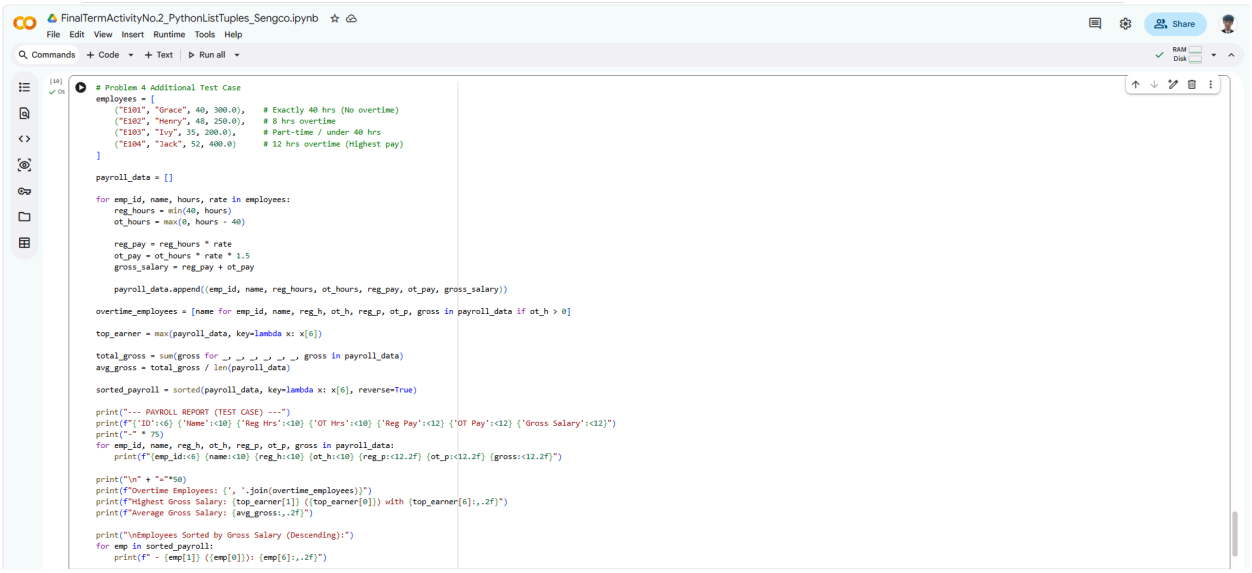
==================================================
Overtime Employees: Brian, David, Ella
Highest Gross Salary: David (E004) with 19,250.00
Average Gross Salary: 12,972.00

Employees Sorted by Gross Salary (Descending):
 - David (E004): 19,250.00
 - Brian (E002): 14,250.00
 - Ella (E005): 13,760.00
 - Alice (E001): 10,000.00
 - Clara (E003): 7,600.00

==================================================
Enter Employee ID to search (e.g., E002): E002

Employee Payroll Details:
  ID:             E002
  Name:           Brian
  Regular Hours:  40
  Overtime Hours: 5
  Regular Pay:    12,000.00
  Overtime Pay:   2,250.00
  Gross Salary:   14,250.00
```

Problem 4: Program Output



```
# Problem 4 Additional Test Case
employees = [
    ("E101", "Grace", 40, 300.0),    # Exactly 40 hrs (No overtime)
    ("E102", "Henry", 48, 250.0),    # 8 hrs overtime
    ("E103", "Ivy", 35, 200.0),      # Part-time / under 40 hrs
    ("E104", "Jack", 52, 400.0)      # 12 hrs overtime (Highest pay)
]

payroll_data = []

for emp_id, name, hours, rate in employees:
    reg_hours = min(40, hours)
    ot_hours = max(0, hours - 40)

    reg_pay = reg_hours * rate
    ot_pay = ot_hours * rate * 1.5
    gross_salary = reg_pay + ot_pay

    payroll_data.append((emp_id, name, reg_hours, ot_hours, reg_pay, ot_pay, gross_salary))

overtime_employees = [name for emp_id, name, reg_h, ot_h, reg_p, ot_p, gross in payroll_data if ot_h > 0]

top_earner = max(payroll_data, key=lambda x: x[6])

total_gross = sum(gross for _, _, _, _, _, _, gross in payroll_data)
avg_gross = total_gross / len(payroll_data)

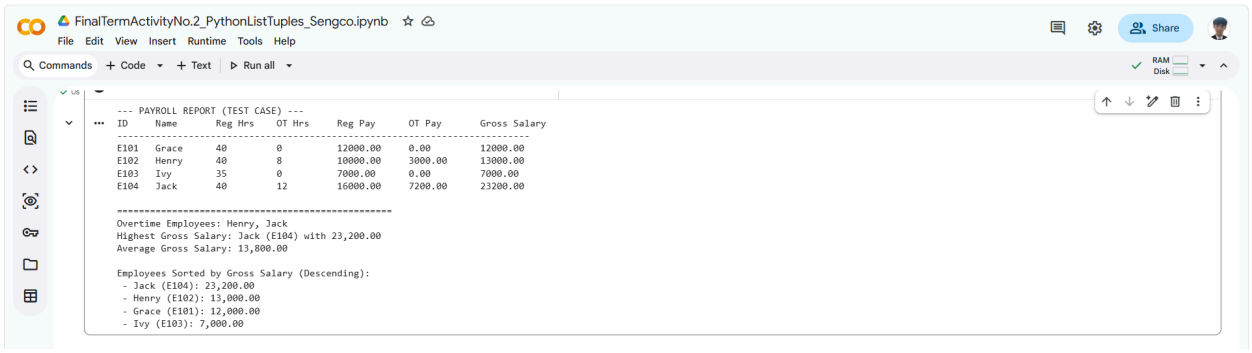
sorted_payroll = sorted(payroll_data, key=lambda x: x[6], reverse=True)

print("--- PAYROLL REPORT (TEST CASE) ---")
print(f"{'ID':<6} {'Name':<10} {'Reg Hrs':<10} {'OT Hrs':<10} {'Reg Pay':<12} {'OT Pay':<12} {'Gross Salary':<12}")
print("-" * 75)
for emp_id, name, reg_h, ot_h, reg_p, ot_p, gross in payroll_data:
    print(f"{emp_id:<6} {name:<10} {reg_h:<10} {ot_h:<10} {reg_p:<12.2f} {ot_p:<12.2f} {gross:<12.2f}")

print("\n" + "="*50)
print(f"Overtime Employees: {', '.join(overtime_employees)}")
print(f"Highest Gross Salary: {top_earner[1]} ({top_earner[0]}) with {top_earner[6]:,.2f}")
print(f"Average Gross Salary: {avg_gross:,.2f}")

print("\nEmployees Sorted by Gross Salary (Descending):")
for emp in sorted_payroll:
    print(f" - {emp[1]} ({emp[0]}): {emp[6]:,.2f}")
```

Problem 4: Additional Test Case Source Code



```
--- PAYROLL REPORT (TEST CASE) ---
ID     Name       Reg Hrs    OT Hrs     Reg Pay      OT Pay       Gross Salary
---------------------------------------------------------------------------
E101   Grace      40         0          12000.00     0.00         12000.00
E102   Henry      40         8          10000.00     3000.00      13000.00
E103   Ivy        35         0          7000.00      0.00         7000.00
E104   Jack       40         12         16000.00     7200.00      23200.00

==================================================
Overtime Employees: Henry, Jack
Highest Gross Salary: Jack (E104) with 23,200.00
Average Gross Salary: 13,800.00

Employees Sorted by Gross Salary (Descending):
 - Jack (E104): 23,200.00
 - Henry (E102): 13,000.00
 - Grace (E101): 12,000.00
 - Ivy (E103): 7,000.00
```

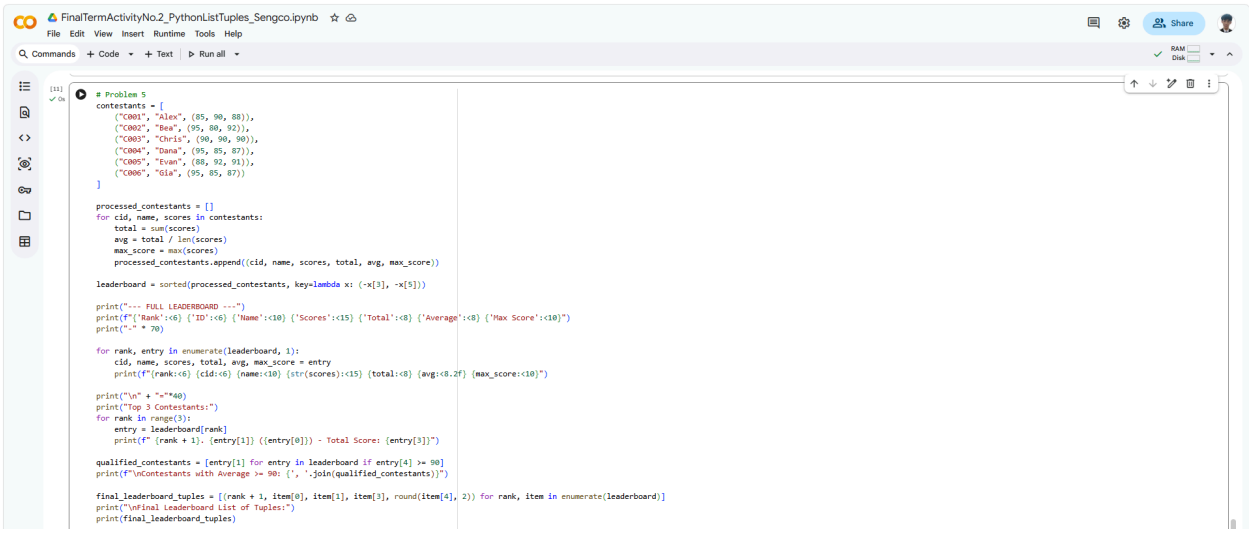
Problem 4: Additional Test Case Program Output

Problem 5: Leaderboard Ranking System

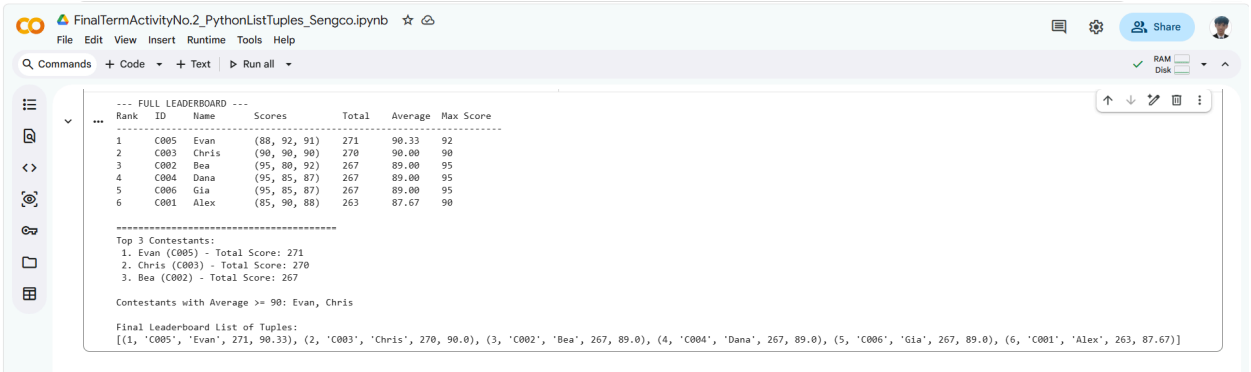
Requirements:

1. Calculate the total and average score of each contestant.
2. Rank contestants by total score from highest to lowest.
3. If two contestants have the same total score, rank them by their highest individual problem score.
4. If both criteria are equal, preserve their original order.
5. Store the final leaderboard as a list of tuples.
6. Display the top three contestants.
7. Identify contestants whose average score is at least 90.

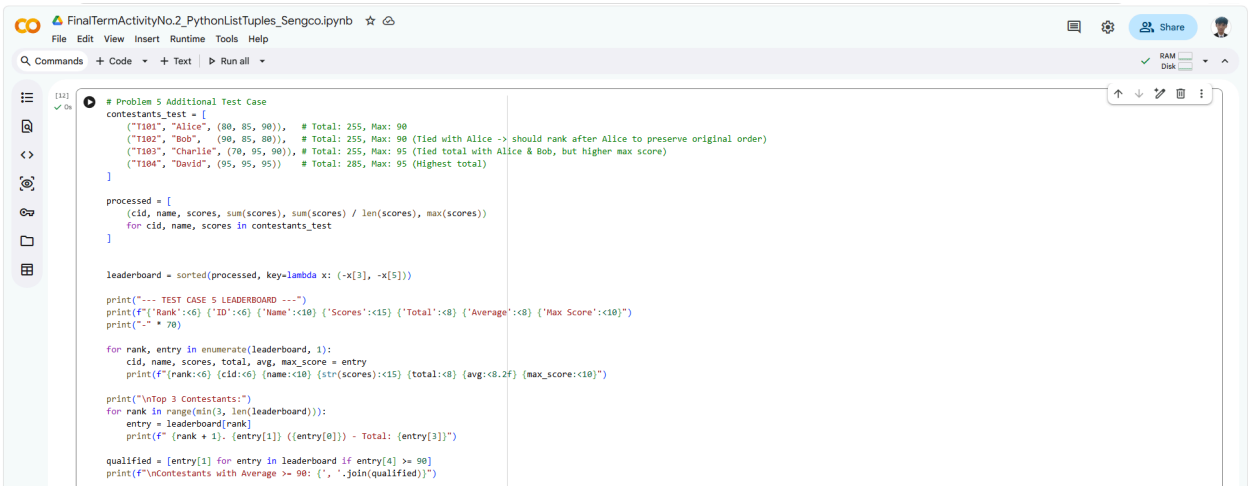
Challenge: Implement the ranking logic using a custom sorting function or sorting key. Do not modify the



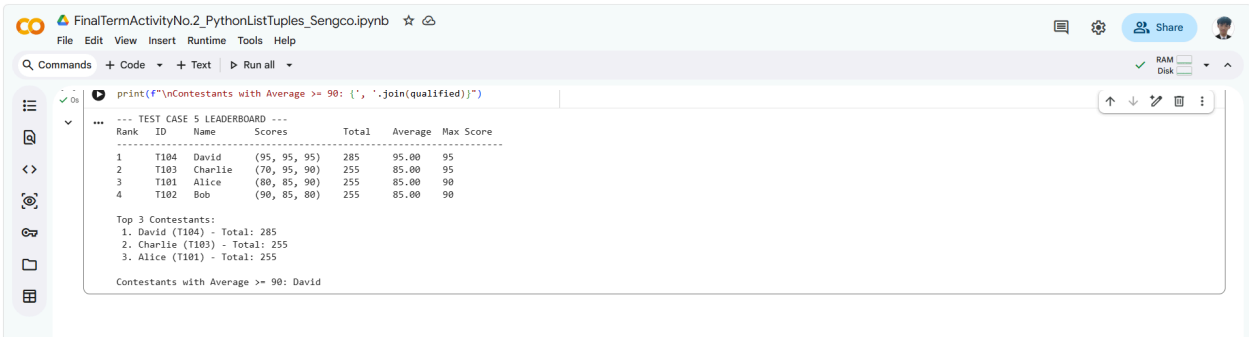
Problem 5: Source Code



Problem 5: Program Output



Problem 5: Additional Test Case Source Code

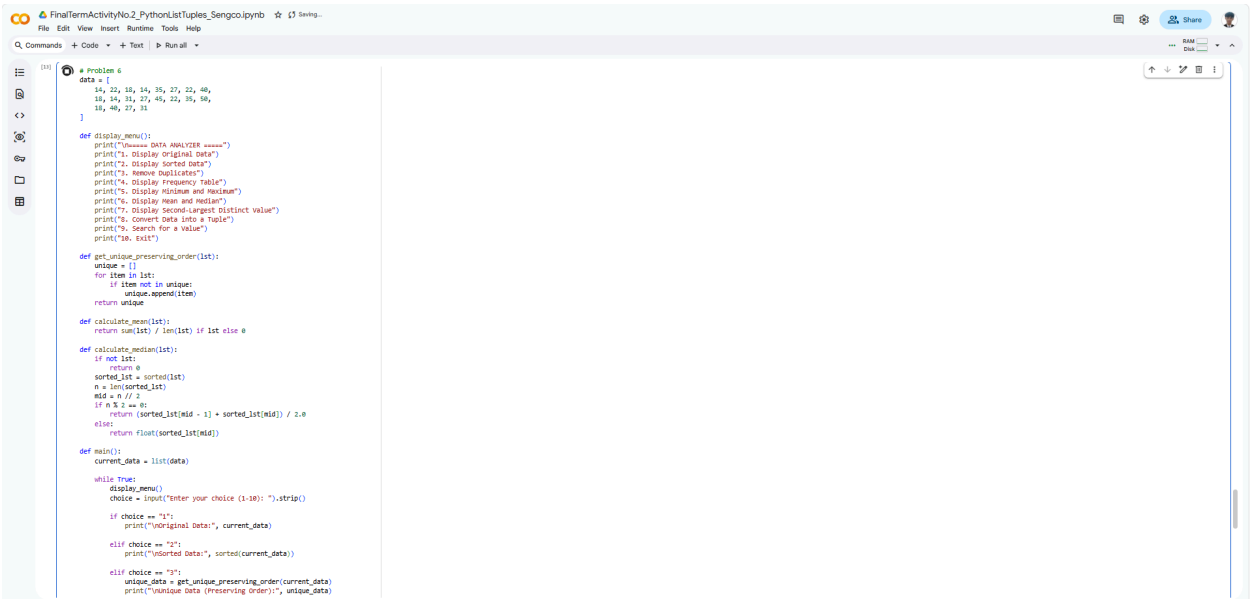


Problem 5: Additional Test Case Program Output

Problem 6: Leaderboard Ranking System

```
Plain text

===== DATA ANALYZER =====
1. Display Original Data
2. Display Sorted Data
3. Remove Duplicates
4. Display Frequency Table
5. Display Minimum and Maximum
6. Display Mean and Median
7. Display Second-Largest Distinct Value
8. Convert Data into a Tuple
9. Search for a Value
10. Exit
```



```
# Problem 6
data = [
    14, 22, 18, 14, 35, 27, 22, 40,
    18, 14, 31, 27, 45, 22, 35, 50,
    18, 40, 27, 31
]

def display_menu():
    print("\n===== DATA ANALYZER =====")
    print("1. Display Original Data")
    print("2. Display Sorted Data")
    print("3. Remove Duplicates")
    print("4. Display Frequency Table")
    print("5. Display Minimum and Maximum")
    print("6. Display Mean and Median")
    print("7. Display Second-Largest Distinct Value")
    print("8. Convert Data into a Tuple")
    print("9. Search for a Value")
    print("10. Exit")

def get_unique_preserving_order(lst):
    unique = []
    for item in lst:
        if item not in unique:
            unique.append(item)
    return unique

def calculate_mean(lst):
    return sum(lst) / len(lst) if lst else 0

def calculate_median(lst):
    if not lst:
        return 0
    sorted_lst = sorted(lst)
    n = len(sorted_lst)
    mid = n // 2
    if n % 2 == 0:
        return (sorted_lst[mid - 1] + sorted_lst[mid]) / 2.0
    else:
        return float(sorted_lst[mid])

def main():
    current_data = list(data)

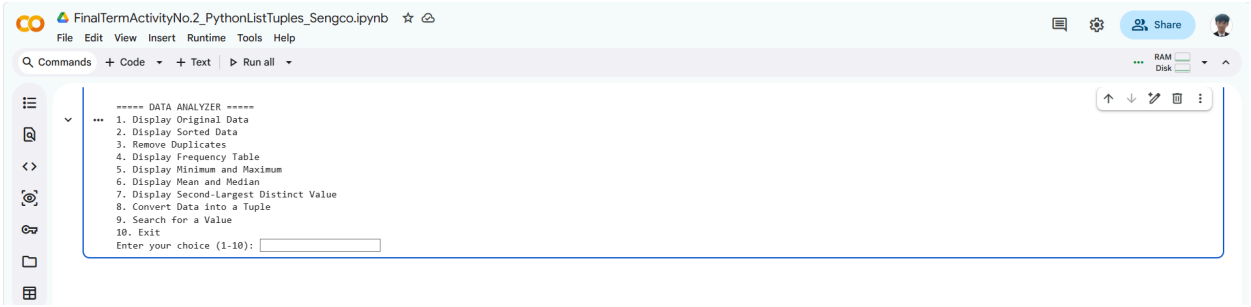
    while True:
        display_menu()
        choice = input("Enter your choice (1-10): ").strip()

        if choice == "1":
            print("\nOriginal Data:", current_data)

        elif choice == "2":
            print("\nSorted Data:", sorted(current_data))

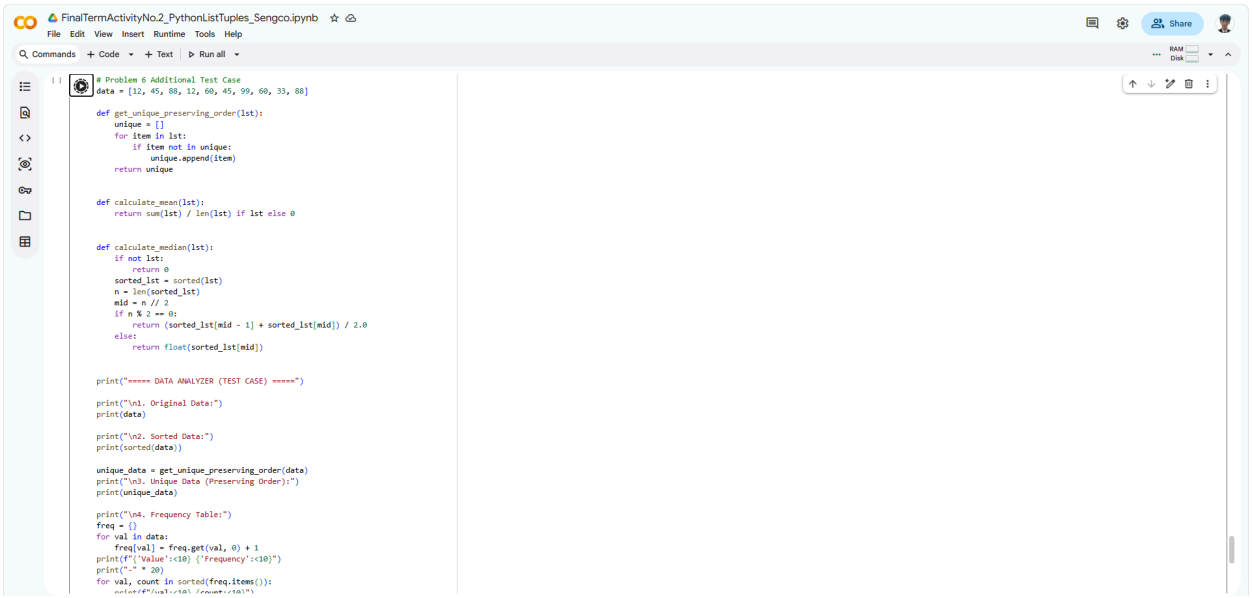
        elif choice == "3":
            unique_data = get_unique_preserving_order(current_data)
            print("\nUnique Data (Preserving Order):", unique_data)
```

Problem 6: Source Code

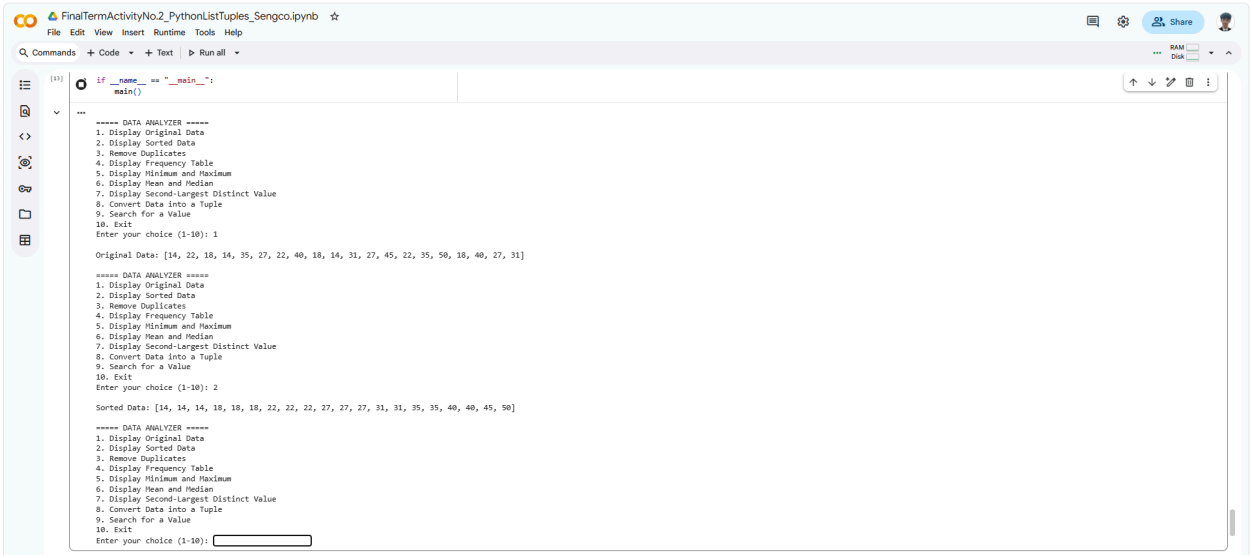


```
===== DATA ANALYZER =====
1. Display Original Data
2. Display Sorted Data
3. Remove Duplicates
4. Display Frequency Table
5. Display Minimum and Maximum
6. Display Mean and Median
7. Display Second-Largest Distinct Value
8. Convert Data into a Tuple
9. Search for a Value
10. Exit
Enter your choice (1-10):
```

Problem 6: Program Output



Problem 6: Additional Test Case Source Code



Problem 6: Additional Test Case Program Output